

Project Report

Meal Management System



Group Members: Arindam Datta(2201011015)

Mohammad Imtiaz Aziz(220101015)

Submitted to: Dr. Muhammed Jamshed Alam Patwary,

Associate Professor,RMSTU.

Contains:

1.Introduction

- 1.1 Project Overview
- 1.2 Problem Statement
- 1.3 Objectives

2.System Analysis & Requirements

- 2.1 User Roles
- 2.2 Functional Requirements

3.Database Design

- 3.1 Entity Relationship (ER)
- 3.2 Schema Structure

4.Implementation:

- 4.1 Technology Stack
- 4.2 System Interfaces

5.Project structure

6.How the monthly calculation follows your pseudo-code

7.How to run

8. Screenshots

- 8.1 Login Interface
- 8.2 before and after meal call tables
- 8.3 meal report

9.Conclusion

- 9.1 Summary
- 9.2 Future Improvements

10.Bibliography

1.Introduction

1.1 Project Overview

This project implements a **Meal Management System** for student mess (canteen) management. It tracks how many meals each student takes before and after the meal call, records money given by students, records expenses made by the manager (admin), and computes meal rate and each student's payable/receivable balances for a selected month.

1.2 Problem Statement

Manual record-keeping of meals and money often leads to mistakes and disputes. The system automates counting, tracks contributions and expenses, and produces a fair monthly settlement for each student.

1.3 Objectives

- Provide secure login for the meal manager/admin.
- Manage student records (add / delete / list).
- Record meals (before-call and after-call) by date.
- Record money given by students and expenses made by the manager.
- Compute monthly meal rate and per-student settlements.
- Provide UI (Swing) for easy operation and NetBeans + Maven-based structure.

2.System Analysis & Requirements

2.1 User Roles

- **Manager (admin / meal_manager):** Login, manage students, record meals, expenses, collect money, and generate monthly settlements.

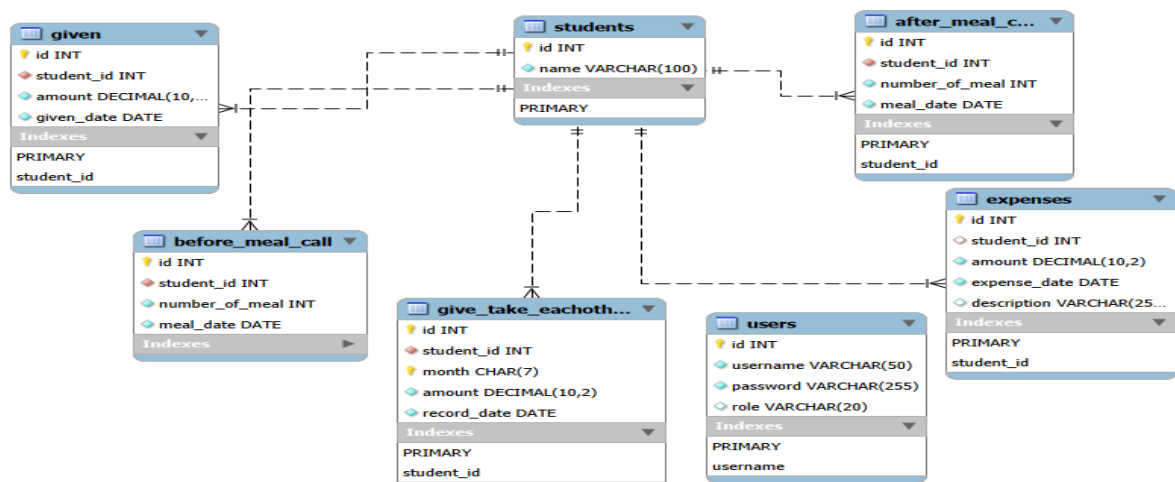
No separate student login in this simple implementation. The manager does all the operations.

2.2 Functional Requirements

- Authentication against users table.
- CRUD for students.
- CRUD for before_meal_call and after_meal_call records.
- CRUD for given (amounts provided by students).
- CRUD for expenses (manager spendings).
- Monthly selection (YYYY-MM) to compute meal rate, balances and settlement list.
- Show list of students who did NOT contribute in a selected month.

3.Database Design

3.1 Entity Relationship Diagram



3.2 Schema Structure

USER

Stores login credentials for the meal manager

- user_id (**PK**)
- username (**UNIQUE**)
- password
- role

Functional Dependency:

user_id → username, password, role

STUDENT

Stores registered students in the meal system.

- student_id (**PK**)
- name

Functional Dependency:

student_id → name

BEFORE_MEAL_CALL

Stores meal count recorded **before** the meal call.

- before_id (**PK**)
- student_id (**FK** → **STUDENT.student_id**)
- meal_date
- number_of_meal

Functional Dependency:

before_id → student_id, meal_date, number_of_meal

Constraint:

- A student can have **at most one before-call entry per day**

AFTER_MEAL_CALL

Stores meal count recorded **after** the meal call.

- after_id (**PK**)
- student_id (**FK** → **STUDENT.student_id**)
- meal_date
- number_of_meal

Functional Dependency:

after_id → student_id, meal_date, number_of_meal

Constraint:

- A student can have **at most one after-call entry per day**

GIVEN

Stores money contributed by students.

- given_id (**PK**)
- student_id (**FK** → **STUDENT.student_id**)
- amount
- given_date

Functional Dependency:

given_id → student_id, amount, given_date

EXPENSES

Stores money spent by the meal manager.

- expense_id (**PK**)
- amount
- expense_date
- description

Functional Dependency:

expense_id → amount, expense_date, description

MONTHLY_SETTLEMENT

Stores final payable or receivable amount for each student per month.

- settlement_id (**PK**)
- student_id (**FK** → **STUDENT.student_id**)
- month (YYYY-MM)
- total_meals
- total_given
- should_pay
- balance
- record_date

Functional Dependency:

settlement_id → student_id, month, total_meals,
total_given, should_pay, balance, record_date

Semantic Meaning of balance:

- balance > 0 → student will **get money**
- balance < 0 → student will **pay money**
- balance = 0 → student is **settled**

4.Implementation & Screenshots

4.1 Technology Stack

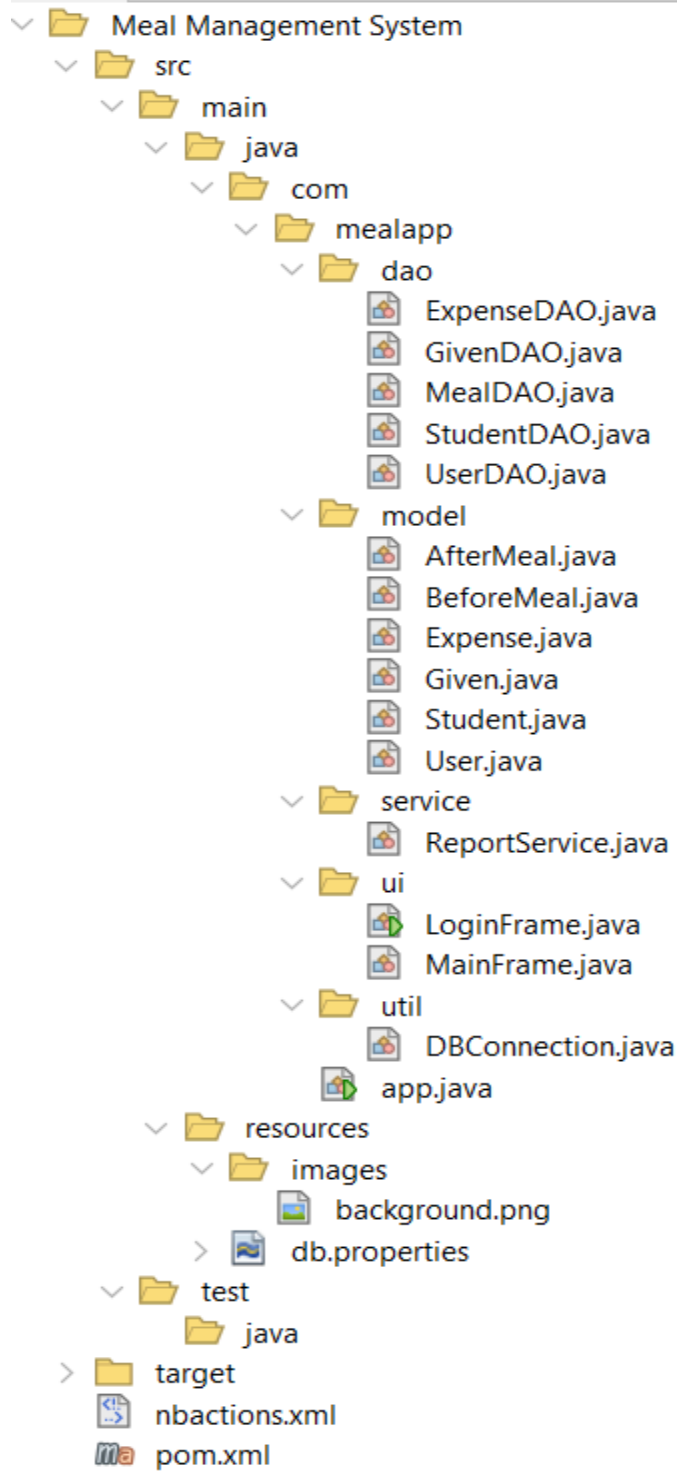
- Java 25 (JDK)
- Swing (Java built-in) for GUI
- Maven to build and manage dependencies
- MySQL 8+ as database
- JDBC driver: mysql-connector-java

4.2 System Interfaces

The desktop application (Swing) includes:

- **Login Interface** — username/password validated against users.
 - If credentials are wrong -> show dialog “Wrong username/password”.
- **Main Dashboard** — Tabbed interface:
 - Students (Add / Delete / List)
 - Before-Call (Add / Delete / List by date)
 - After-Call (Add / Delete / List by date)
 - Given (Add / Delete / List by date)
 - Expenses (Add / Delete / List by date)
 - Reports: pick month (YYYY-MM) -> compute meal rate, show list of students with who will pay or get money >> compute $\text{meal_rate} = \text{total_expenses} / \text{total_meal}$ using the algorithm you gave (max of before/after per student per day).

5. Project structure



6.How the monthly calculation follows your pseudo-code

- For each day of selected month:
 - For each student, compute `final[day][student] = max(before, after)`.
- Compute `total_meal_per_student[student]` and `total_meal`.
- Sum `total_expenses` from expenses for that month.
- `meal_rate = total_expenses / total_meal` (guarding divide-by-zero).
- For each student compute `should_pay = total_meal_per_student[i] * meal_rate`, `balance = expenses_given_by_student - should_pay` and categorize output as “will get / will pay / is settled”.

The implementation in `MealService` follows this logic.

7.How to run (NetBeans / Maven)

- Create database: run the SQL file provided earlier in your MySQL client.

- Update `src/main/resources/db.properties` with your DB credentials.
- In NetBeans: `File -> Open Project` and open the folder with `pom.xml`. NetBeans will import the Maven project.
- Build & Run (`Run -> Run Project`) or right-click `App.java` and run. The login window appears. Use the inserted demo credentials:
 - username: `meal_manager`
 - password: `password123`
- Use GUI to add students, record before/after meals, given amounts and expenses, then go to Reports tab and compute for YYYY-MM (e.g., 2026-02).

8.Screenshots:

8.1 Login Interface:



8.2 Student Names:

Meal Management System

Students Expenses Before Meal After Meal Given Reports Not Given Students

Name: Add Delete

ID	Name
6	arindam
5	Hasan
2	Karim
4	Nayeem
1	Rahim
3	Sumon

8.3 before and after meal call tables:

Meal Management System

Students Expenses Before Meal After Meal Given Reports Not Given Students

Student: arindam Meals: Date (YYYY-MM-DD): Add Delete

ID	Student	Meals	Date
1	Rahim	2	2026-01-01
2	Karim	1	2026-01-01
3	Sumon	2	2026-01-01
4	Nayeem	1	2026-01-01
5	Hasan	2	2026-01-01
6	Rahim	1	2026-01-02
7	Karim	2	2026-01-02
8	Sumon	1	2026-01-02

Meal Management System

Students Expenses Before Meal After Meal Given Reports Not Given Students

Student: arindam Meals: Date (YYYY-MM-DD): Add Delete

ID	Student	Meals	Date
1	Rahim	1	2026-01-01
2	Karim	2	2026-01-01
3	Sumon	1	2026-01-01
4	Nayeem	2	2026-01-01
5	Hasan	1	2026-01-01
6	Rahim	2	2026-01-02
7	Karim	1	2026-01-02
8	Sumon	2	2026-01-02
9	Nayeem	1	2026-01-02

8.4 meal report:

=== Meal Report ===

From: 2026-01-01 To: 2026-01-30

Total Expenses: \$76,175.00 Total Meals: 300 Meal Rate: \$253.92

Student	Meals	Rate	Should Pay	Paid	Balance	Avg/Day
arindam	0	\$253.92	\$0.00	\$0.00	settled	0.00
Hasan	60	\$253.92	\$15,235.20	\$6,900.00	-\$8,335.20	2.00
Karim	60	\$253.92	\$15,235.20	\$10,400.00	-\$4,835.20	2.00
Nayeem	60	\$253.92	\$15,235.20	\$7,200.00	-\$8,035.20	2.00
Rahim	60	\$253.92	\$15,235.20	\$6,600.00	-\$8,635.20	2.00
Sumon	60	\$253.92	\$15,235.20	\$7,600.00	-\$7,635.20	2.00
TOTAL	300	\$253.92	\$76,176.00	\$38,700.00	-\$37,476.00	10.00

Students who didn't contribute:
- arindam

Report generated on: 2026-02-10

9.Conclusion

9.1 summary

I developed a **Meal Management System** using **Java (Swing GUI)** and **MySQL**, designed to manage daily and monthly meal records for bachelor mess management or shared living environments. The system provides a complete solution for tracking meals, expenses, and user data with proper authentication and reporting. The application includes a **secure login system** that validates users against a database. Users can record daily meals, manage members, track expenses, and generate **date-based and month-based reports**. All data is stored persistently in a MySQL database, ensuring accuracy and reliability. The project follows a **structured, modular design** using DAO and service layers, making it scalable and maintainable. Additional features include report generation and export (PDF), GUI-based interaction, and proper exception handling. This project demonstrates my understanding of **Java OOP concepts, Swing-based desktop application development, database design, SQL integration**, and **real-world problem solving**. It also reflects my ability to build end-to-end software solutions rather than isolated academic programs.

9.2 Future Improvements

- Use password hashing (bcrypt).
- Add user roles & permissions, audit logs.
- Replace Swing with JavaFX or a web UI for better UX.
- Add reporting exports (CSV, PDF).
- Add unit tests and validation.
- Add concurrency-safe DB operations and transactions.

10.Bibliography

- Official MySQL docs (for schema design and constraints)
- Oracle Java docs (Swing, JDBC)